

Projet Bases de Données (BD6)

Le Big Match : Big Brother Is Matching You

Le projet s'effectue par groupe de 2 personnes (qui peuvent appartenir à des groupes de TP différents). **Les questions concernant le projet doivent être posées sur le discord dédié et non par mail.** La récupération de données existantes (e.g., événements, lieux réels) est non seulement autorisée, mais encouragée. En revanche, les règles anti-plagiat de l'Université s'appliquent à votre travail et toute utilisation d'éléments extérieurs devra être explicitement évoquée et sourcée. Vous pouvez discuter des différents aspects du projet avec les autres groupes, mais toute copie partielle ou totale de code entre plusieurs groupes est interdite.

Rendu 1 : Modélisation conceptuelle

Date : TBA.
Format : Schéma entités-associations.
Consignes : Réalisez le modèle conceptuel de données (schéma entités-associations) de votre base de données.

Rendu 2 : Code

Date : TBA.
Format : Une *unique* archive contenant l'ensemble de vos fichiers de code et un fichier **README.txt** expliquant brièvement à quoi correspond chaque fichier, déposée sur Moodle. Pour créer une archive **MonArchive.tar** à partir d'un dossier **MonCode** utilisez la commande Linux : `tar cvf MonArchive.tar MonCode`
Consignes : Implémentez dans **PostgreSQL** votre base de données et les requêtes permettant de l'utiliser, en utilisant la modélisation effectuée pour le Rendu 1 et les consignes détaillées à partir de la page 2.

Rendu 2bis : Rapport

Date : TBA.
Format : Rapport entre 4 et 8 pages au format PDF déposé sur Moodle. Vous pouvez utiliser <https://www.freepdfconvert.com/fr> pour convertir en PDF.
Consignes : Vous incluez votre modèle entités-associations et un schéma relationnel de vos tables. Vous expliquerez les choix qui ont motivé votre modélisation, et les limitations de votre modèle. Vous décrirez également les requêtes que vous avez choisies d'implémenter, et les choix que vous avez faits pour le calcul des indices de recommandation (voir Section 4).

Rendu 3 : Soutenance

Date : TBA.
Format : Présentation orale par *tous* les membres du groupe.
Consignes : Vous débutez votre soutenance par une présentation de votre travail. Nous vous poserons ensuite des questions sur vos choix de modélisation, les contraintes d'intégrité utilisées, les points forts et les points faibles de votre approche, le fonctionnement de vos requêtes, etc.

1 Le Big Match : Big Brother Is Matching You

D'après une infographie basée sur les chiffres d'une étude de l'université de Stanford et partagée sur les réseaux sociaux par le créateur de contenu James Eagle, aujourd'hui 60,76 % des couples se sont rencontrés en ligne, par le biais d'une application de rencontre ou d'un réseau social. Cependant, bien souvent, l'excitation des premiers échanges virtuels ne survit pas aux rencontres en chair et en os. Sans lendemain, les premiers rendez-vous embarrassants s'enchaînent, alors même que l'attraction ne survit souvent même pas au premier coup d'oeil. Dans ce contexte, Le Big Match vous propose grâce à vos traces numériques de revenir à la réalité et de trouver l'amour lors de véritables événements pendant lesquels vous vous livrez à de véritables activités. Le principe est simple : vous autorisez l'application à accéder à l'ensemble de vos traces numériques et celle-ci se charge de vous suggérer les événements lors desquels vous serez le plus susceptible de trouver l'âme soeur. Un système de notification discrète est même proposé lorsque vous passez près d'un match potentiel. Dans ce projet, nous vous proposons de participer à la création de la base de données qui sera utilisée par Le Big Match.

Pensé comme "the social network to rule them all", Le Big Match collectera, intégrera et agrégera l'ensemble des informations que vous laissez sur internet et sur les réseaux : des concerts auxquels vous avez indiqué vous rendre sur facebook à vos sorties vélos ou runs strava passés ou à venir, en passant par les réels auxquels vous avez réagi ou les personnes que vous suivez sur instagram. Tout pourra être utilisé pour vous faire trouver l'amour, jusqu'à vos historiques d'achat sur Amazon, Vinted ou vos consultations de recettes de cuisine sur Marmiton, rédactions d'avis sur Dicogs et achats de billets sur Ticketmaster. Il n'est évidemment pas raisonnable dans ce projet de fournir une modélisation exhaustive : vous choisirez de vous concentrer sur les données provenant d'au moins deux réseaux sociaux ou sites web. Il faudra a minima que ces données vous permettent de récupérer des événements à venir et futurs grâce auxquels vous pourrez par la suite construire vos recommandations.

Les utilisateurs du réseau social seront des personnes majeures qui seront identifiées par leur pseudo et pourront remplir un ensemble de formulaires types quant à leurs caractéristiques : ville et pays de résidence, genre, orientation, taille, poids, couleur des yeux, etc. Veillez à modéliser le genre et l'orientation de manière réaliste et non comme des notions binaires. Comme sur tinder, il sera possible à un utilisateur de liker ou noper un autre utilisateur. Lorsque deux utilisateurs se likent mutuellement : c'est un match ! Attention, comme sur tinder, seuls les abonnés peuvent annuler un like ou un nope. Évidemment, la recommandation d'événements tiendra compte pour un utilisateur donné à la fois du nombre de matchs effectifs et du nombre de matchs potentiel présents à l'évènement (à vous de définir la notion de match potentiel grâce à votre algorithme de recommandation!).

Il sera également possible pour les utilisateurs d'annoncer des événements directement sur Le Big Match. Les caractéristiques de ces événements pourront être listées (description, lieu, prix, organisateurs, mots clefs, etc.). Les utilisateurs pourront indiquer être intéressés ou participer à un évènement (mais pas les deux à la fois). Les utilisateurs pourront taguer leur propre profil avec les mêmes mots clefs que ceux utilisés pour taguer les événements. Certains lieux (comme les cinémas, les musées, les salles de sport, de concert ou les clubs) pourront également être tagués. N'importe quel mot pourra être choisi (sur le modèle des hashtags de Instagram ou Twitter), mais il est envisageable de construire un système de catégories et sous-catégories (à votre discrétion).

Plusieurs services de consultation d'historique ou d'affichage de suggestions seront proposés aux utilisateurs (événements auxquels ils ont assistés ou auxquels ils pourraient assister, matchs effectifs ou potentiels qu'ils ont croisé ou pourraient croiser lors d'événements,...), l'offre étant bien sûr plus large pour les abonnés.

Concevez une base de données qui permette de gérer Le Big Match. Si vous le souhaitez, vous pouvez vous inspirer des applications de rencontre existantes pour ajouter quelques fonctionnalités (service de messagerie par exemple), mais aussi ajouter vos propres fonctionnalités. Réfléchissez au préalable aux requêtes que vous serez amenés à implémenter (Section 3 et 4) pour savoir quelles informations seront indispensables dans votre base. Vous commencerez par réaliser le modèle conceptuel de données (schéma entités-associations) pour le Rendu 1, puis vous pourrez passer à l'implémentation dans PostgreSQL pour le Rendu 2. Utilisez les différents outils étudiés en cours et en TP (clés primaires/étrangères, contraintes d'intégrité, types adaptés ...). Expliquez dans le rapport (Rendu 2bis) et lors de la soutenance (Rendu 3) comment vous avez conçu cette base de données, les choix que vous avez effectués, les avantages, limites et améliorations possibles de votre modèle, etc. Dégagez également les problèmes éthiques et légaux soulevés par une telle application. Quels garde-fous pensez-vous pouvoir proposer ?

Prenez bien garde à dégager les contraintes externes, i.e., veillez à bien séparer ce qui relèvera du SGBD et ce qui relèvera de l'application et précisez-le dans le rapport. Il est normal que vous ne puissiez pas implémenter toutes les contraintes externes au niveau du SGBD avec ce que nous avons appris pour le moment. Vous êtes libres d'implémenter des triggers si vous le souhaitez, mais ce n'est pas demandé et pas au programme de cette année.

Aide à l'utilisation de PostgreSQL

En plus de vos cours et TPs, vous êtes encouragés à consulter la documentation en français de PostgreSQL disponible ici :

<https://docs.postgresql.fr/current/>

Les sections I. Tutoriel (<https://docs.postgresql.fr/current/tutorial.html>) et II. Langage SQL (<https://docs.postgresql.fr/current/>) sont les plus pertinentes. Il y est expliqué comment compter, calculer un maximum, ajouter des contraintes lors de la création d'une table (empêcher un attribut d'être négatif par exemple), faire des opérations mathématiques, etc.

Vous pouvez également consulter le fichier **Installation et commandes basiques pour PostgreSQL** disponibles sur Moodle :

<https://moodle.u-paris.fr/course/view.php?id=1655>

2 Peuplez vos tables

Maintenant que vos tables sont créées, vous pouvez les remplir avec les informations adaptées (profils d'utilisateurs, matches, événements...). Insérez au moins une centaine de tuples au total dans votre base de données. Les tables seront alimentées à partir de fichiers csv. Vous mettrez tous les fichiers csv utilisés (ou un fichier txt avec des liens permettant d'y accéder en ligne s'ils sont trop volumineux) dans un répertoire nommé CSV afin de ne pas les mélanger avec les scripts sql. Vous pourrez alimenter vos tables avec la technique qui consiste à :

- importer les fichiers csv avec COPY dans des tables temporaires,
- « oublier » les attributs inutiles en supprimant les colonnes via `ALTER TABLE ... DROP COLUMN`,
- alimenter vos tables à partir de ces tables temporaires, par exemple former des couples nom, prénom à partir de deux tables, une avec des prénoms et une autres avec des noms.

Bien sûr le travail le plus ingrat consiste à produire les fichiers csv avec les données. Vous pouvez par exemple écrire dans votre langage de programmation préféré (Java, Python, ...) une boucle for qui affiche des tuples générés automatiquement (avec la syntaxe attendue par PostgreSQL), et les recopier ensuite dans votre code PostgreSQL.

Vous pouvez aussi utiliser directement les structures de contrôle intégrées à PostgreSQL :

<https://docs.postgresql.fr/current/plpgsql-control-structures.html>

Vous pouvez également utiliser des outils de génération de données en ligne tels que :

- <https://www.mockaroo.com/>,
- <https://generatedata.com/>,
- etc.

Les plus courageux peuvent récupérer des données réelles. Vous pouvez par exemple consulter :

- <https://www.kaggle.com/datasets/andrewmvd/okcupid-profiles>
- <https://www.kaggle.com/datasets/rabieelkharoua/predict-online-dating-matches-dataset>
- <https://www.kaggle.com/datasets/ashleyxu98/tinder>
- <https://www.kaggle.com/datasets?search=music>

- <https://www.kaggle.com/datasets?search=concert>
- <https://www.kaggle.com/search?q=spotify>
- <https://www.kaggle.com/datasets/gorororororo23/aereregre>
- <https://data.world/datasets/music>
- etc.

Attention, il est irréaliste de trouver sur internet des fichiers parfaitement prêts à être utilisés tels quels. La récupération et le nettoyage de vraies données peut être un exercice fastidieux et se fait d'habitude au moyen de bibliothèques spécialisées (R et Python sont les langages typiquement utilisés pour ces tâches, voir par exemple <https://www.simplilearn.com/tutorials/data-analytics-tutorial/spotify-data-analysis-project>). Attention notamment au format des données. Si vous souhaitez vraiment récupérer de vraies données, le plus simple sera probablement de récupérer des données au format csv.

3 Effectuez des requêtes

Il est maintenant temps d'utiliser votre base de données pour faire vivre votre application de rencontre. Imaginez 20 questions sur la base de données que vous avez modélisée, et écrivez des requêtes SQL permettant d'y répondre. Vous pourrez éventuellement utiliser ces requêtes pour vos algorithmes de matching (partie 4 du sujet). L'originalité des questions, leur pertinence pour l'application et la difficulté des requêtes (si tant est que celle-ci soit nécessaire) seront prises en compte dans la notation. Parmi vos requêtes, il faut au minimum :

- une requête qui porte sur au moins trois tables ;
- une 'auto jointure' ou 'jointure réflexive' (jointure de deux copies d'une même table)
- une sous-requête corrélée ;
- une sous-requête dans le FROM ;
- une sous-requête dans le WHERE ;
- deux agrégats nécessitant GROUP BY et HAVING ;
- une requête impliquant le calcul de deux agrégats (par exemple, les moyennes d'un ensemble de maximums) ;
- une jointure externe (LEFT JOIN, RIGHT JOIN ou FULL JOIN) ;
- deux requêtes équivalentes exprimant une condition de totalité, l'une avec des sous requêtes corrélées et l'autre avec de l'agrégation ;
- deux requêtes qui renverraient le même résultat si vos tables ne contenaient pas de nulls, mais qui renvoient des résultats différents ici (vos données devront donc contenir quelques nulls), vous proposerez également de petites modifications de vos requêtes (dans l'esprit de ce qui sera présenté dans le cours sur l'information incomplète) afin qu'elles retournent le même résultat ;
- une requête récursive (par exemple, une requête permettant de calculer quelle est la prochaine date à laquelle un lieu très demandé est disponible pour l'organisation d'un nouvel événement) ;
- une requête utilisant du fenêtrage (par exemple, pour chaque mois de 2024, le pseudo et les mots clefs associées aux dix personnes qui ont été notées le plus souvent ce mois-ci).

Prenez l'habitude de bien programmer vos requêtes SQL. Structurez vos requêtes et surtout indentez-les correctement. N'utilisez pas les sous-requêtes là où une jointure suffirait. N'utilisez pas les vues si c'est uniquement dans le but de simplifier l'écriture d'une requête complexe. Enfin, vous pouvez essayer de trouver une requête qui vous semble difficile (ou impossible) à effectuer dans votre modèle, et suggérer une modification de votre modélisation initiale qui faciliterait l'implémentation de cette requête.

4 Vers un algorithme de matching

La force des applications de rencontre modernes, c'est la finesse de leurs algorithmes de matching. Les profils d'utilisateurs les plus susceptibles de vous plaire vous sont proposés en premier. Avec Le Big Match, la révolution c'est que des événements vous sont également suggérés! Si vous êtes une femme hétéro aimant les sports de grand air, alors vous aurez de grandes chances de vous voir proposer une sortie de cyclisme sur route, où il se trouve que les hommes hétéros sont souvent majoritaires (et souvent célibataires, puisqu'ils passent tout leur temps libre à rouler). Ce type de fonctionnalité n'est pas totalement nouveau et vous pouvez vous inspirer d'outils existants. Par exemple, si vous utilisez facebook, vous avez peut-être déjà expérimenté sa fonctionnalité de recommandation d'événements. En fonction de votre lieu de résidence et de l'historique des événements suivis par vous-même et votre réseau, des événements à venir susceptibles de vous intéresser peuvent vous être proposés. Vous vous renseignerez d'abord sur les techniques de filtrage collaboratif utilisées dans les systèmes de recommandation. Vous concevrez ensuite quelques requêtes sur lesquelles vous pourrez baser vos recommandations (que vous pourrez bien sûr proposer dans la section précédente!). Vous pourrez même proposer un indice de recommandation pour chaque profil et événement à venir en fonction de ces requêtes. Vous calculerez cet indice selon la méthode et les critères de votre choix. Vous pourrez ensuite l'utiliser pour aider à la recommandation de profils et d'événements. Ceci pourra par exemple vous permettre de proposer une requête récupérant les n profils ou événements avec l'indice le plus élevé (profils ou événements à suggérer en priorité) pour un utilisateur donné à un moment donné. Expliquez dans le rapport (Rendu 2bis) le fonctionnement de votre indice et les raisons qui vous ont guidées dans votre choix. Détaillez ses points forts, ses limites et comment l'améliorer. Il n'est pas interdit de discuter des aspects éthiques, bien au contraire.

5 Conseils pour la présentation

Pour rendre votre projet plus interactif, vous pouvez préparer des requêtes dont certains paramètres seront fournis par l'utilisateur. Par exemple, afin de retourner les lieux potentiels dans lesquels on peut organiser des événements dans une ville donnée, on pourrait considérer la requête suivante :

```
SELECT lieu
FROM événement
WHERE ville='Paris';
```

Mais comment faire si la ville n'est pas connue d'avance ?

Préparation de requêtes paramétrées Vous pouvez créer une requête paramétrée en utilisant la syntaxe suivante.

```
PREPARE recherche_par_ville(VARCHAR) as
SELECT lieu
FROM concert
WHERE ville=$1;
```

Vous pouvez ajouter autant de paramètres que vous souhaitez. Dans la requête, vous utiliserez \$1, \$2... pour en obtenir la valeur. La requête est préparée mais pas exécutée. Pour l'exécuter, on tape sous psql :

```
EXECUTE recherche_par_ville('Marseille');
```

La requête est alors exécutée avec la valeur 'Marseille' à la place du paramètre \$1.

Variables d'environnement et prompt La commande `prompt` de `psql` permet de faire la saisie au clavier. La valeur est sauvegardée dans une variable d'environnement (`v_ville` dans l'exemple qui suit).

```
\prompt 'Tapez le nom de ville -> ' v_ville
SELECT lieu
FROM concert
WHERE ville = :v_ville;
```

Lorsque la requête est exécutée, `:v_ville` prend la valeur de la variable d'environnement `v_ville`.

Tous ces éléments permettent de faire un script interactif que vous exécuterez au cours de votre présentation.

Manipulation des dates Le type Postgres `DATE` permet de stocker une date et le type `TIME` permet de stocker l'heure. Il existe également un type `TIMESTAMP` qui contient la date et l'heure.

Une chaîne peut être convertie en type `DATE` avec la fonction `to_date(chaine_a_convertir, format)`, où `format` est une chaîne de la forme `'YYYYMMDD'` qui indique le format dans lequel est donnée la date. On peut aussi écrire `date '2015-03-05'` pour obtenir une date. *Attention, selon la configuration de postgres, la date '12-11-2015' peut être interprétée comme le 12 novembre, ou le 11 décembre.*

On peut ajouter un entier `n` à une date pour obtenir la date `n` jours plus tard, ou soustraire une date à une autre pour connaître le nombre de jours entre les deux dates.

Plusieurs fonctions prédéfinies en postgres permettent de manipuler les données de type `DATE`, `TIME`, `TIMESTAMP`.

Fonction	Usage	Exemple
<code>EXTRACT</code>	extraire un jour/mois/... d'une date ou d'un timestamp	<code>EXTRACT (DAY FROM madate)</code>
<code>INTERVAL</code>	préciser une unité de temps (jour/heure/semaine...)	<code>heureRDV + INTERVAL '2 hours'</code>
<code>CURRENT_DATE</code>	Obtenir la date du jour	
<code>CURRENT_TIME</code>	Obtenir l'heure actuelle	

On peut énumérer des valeurs dans un intervalle avec la fonction `generate_series`. Par exemple, `generate_series(1,5)` énumère les valeurs 1, 2, 3, 4, 5. `generate_series(1,5,2)` énumère les valeurs de 1 à 5 avec un pas de 2 : 1, 3, 5. `generate_series(current_date+time '10:00', current_date+time '16:00', interval '30 minutes')` énumère les créneaux de 30 minutes aujourd'hui entre 10h et 16h.